

browser-research-mcp

Browser-based research as an MCP server. Drives a real Chromium via

patched Playwright (`patchright`) so the agent can read JavaScript-rendered tables, dynamic charts, login-walled dashboards, and AJAX dropdowns that the cheaper rungs of the fetch ladder can't reach.

This is the **last rung** of the ladder:

```
web_search → web_fetch → pdf_fetch → http_post_form → browser-research
```

Tools

Tool	Purpose
<code>visit(url, ...)</code>	Open a URL with Chromium, return DOM text + screenshot. Cheap, no LLM call.
<code>act(url, steps, ...)</code>	Drive clicks/fills/selects/ <code>fetch_json</code> through a flow, then Sonnet-extract. Auto-captures the page's XHR/fetch → <code>observed_api</code> + a <code>recovery_hint</code> when a UI step fails.
<code>extract(url, focus, ...)</code>	<code>visit</code> + Sonnet structured extraction. Same response shape as <code>pdf_fetch_structured</code> . Sends the screenshot to Sonnet so chart values drawn via canvas/SVG get picked up.
<code>download_file(url, query?, ...)</code>	Download + parse a <code>.xlsx/.xslm/.xls/.csv/.tsv/.pdf</code> end-to-end. Pass <code>query</code> to grep (pdfgrep-style) and get back only the matching pages/rows + snippets — not the whole 200-page file.
<code>sitemap_probe(url, ...)</code>	Read robots.txt + sitemap(s); surface data-like URLs (<code>.csv/.xlsx/.json/.pdf</code> , <code>/api</code>) to fetch directly. Cheapest discovery step.
<code>inspect_network(url, steps?, ...)</code>	Discover the AJAX endpoint(s) a JS dashboard fires — method, params, response sample.

Tool	Purpose
<code>call_api(url, method, body, ...)</code>	Replay a data endpoint directly from the page's own origin. Reaches data the UI never exposes.
<code>strategy()</code>	Return the decision procedure — escalation ladder + signal→action table + principles.

Adaptiveness, built in

The method isn't left to a doc the agent might skip — it's encoded three ways

(`strategy.py`):

- **Always-loaded instructions** carry a compact APPROACH ladder: classify the page, take the cheapest rung that works (static fetch → `visit` → `act` → `inspect_network` / `call_api` → `download_file` → pivot), and escalate on a *specific signal* rather than retrying what just failed.
- **A per-result advisor** (`diagnose_next`) rides along on every browser-tool result as a `next_step` field, naming the recommended move from the signals it saw — `auth_wall` → use open data; `blocked` → pivot/clean IP; `observed_api` / `recovery_hint` → `call_api`; `file_links` → `download_file`; sparse DOM → re-`visit` / `inspect_network`.
- **The `strategy` tool** returns the full ladder + signal table on demand.

Core principles baked in: look before you assert, probe before you build (verify params on a known period first), prefer API JSON over DOM over OCR, verify totals, then cache the win as a playbook.

API replay — the sharp edge for JS dashboards

Most government dashboards (PPAC, RBI, NSE, MoSPI) render their tables from an AJAX endpoint, fronted by a custom JS dropdown that ordinary click/`select_option` automation can't drive. Rather than fight the widget, discover and replay the endpoint:

```
inspect_network(url, steps=[change the year dropdown])    # → endpoint + params
    ↓
call_api(endpoint, method="POST", body={...templated for the period...}) # → JSON
    ↓
save it as a playbook `api` recipe so the discovery step is skipped next time
```

`call_api` runs `fetch()` *inside* a page on the endpoint's origin, so cookies, CSRF state and `Origin` / `Referer` all match — and it routinely returns periods the dropdown omits (e.g. PPAC natural-gas FY2023-24, which isn't selectable in the UI but the endpoint still serves).

Why patchright

`patchright` is a drop-in Apache-2.0 patched Playwright that disables the `AutomationControlled` blink feature, removes `Runtime.enable` leaks, and a few other detection vectors. Indian government dashboards (PPAC, RBI, MoSPI, SEBI) work fine with this without paying for residential proxies or a hosted browser SaaS.

Run locally

```
uv tool install browser-research-mcp --python 3.12
# Install Chromium for patchright (one-off):
uv tool run patchright install chromium

# stdio (Claude Desktop / Cursor / desktop clients):
ANTHROPIC_API_KEY=... uvx browser-research

# HTTP (the platform backend):
ANTHROPIC_API_KEY=... uvx browser-research --transport streamable-http --port 7862
```

Environment

Var	Required	Default
<code>ANTHROPIC_API_KEY</code>	for <code>extract</code> (not <code>visit</code>); also powers the <code>web_fetch</code> fallback	—
<code>ANTHROPIC_MODEL</code>	no	<code>claude-sonnet-4-6</code>
<code>TAVILY_API_KEY</code>	no — enables the 1st fetch fallback when a CDN bot-blocks Chromium	—

Var	Required	Default
<code>BROWSER_PROXY_SERVER</code>	no — residential/ISP proxy for <code>use_proxy</code> calls (e.g. <code>http://gw.proxy.net:7000</code>); the egress IP is the dominant CDN-block signal	—
<code>BROWSER_PROXY_USERNAME</code> / <code>BROWSER_PROXY_PASSWORD</code>	no — proxy auth	—
<code>BROWSER_ENGINE</code>	no — <code>chromium</code> or <code>camoufox</code> (see below)	<code>chromium</code>
<code>BROWSER_CHANNEL</code>	no — e.g. <code>chrome</code> for a real Google Chrome binary (must be in the image) instead of bundled Chromium	—
<code>HEADLESS</code>	no	<code>true</code> (<code>false</code> headful; <code>virtual</code> = Xvfb, camoufox only)
<code>PLAYBOOKS_GCS_BUCKET</code>	no — GCS bucket for editable, hot-reloaded playbooks	— (uses in-repo defaults)
<code>PLAYBOOKS_GCS_OBJECT</code>	no — object key for the playbooks JSON	<code>config/playbooks.json</code>
<code>PLAYBOOKS_TTL_SECONDS</code>	no — hot-reload interval	<code>60</code>
<code>ADMIN_TOKEN</code>	no — gates the <code>/admin/playbooks</code> API (fail-closed if unset)	—
<code>MCP_TRANSPORT</code>	no	<code>stdio</code>
<code>MCP_HOST</code> / <code>PORT</code>	no	<code>0.0.0.0</code> / <code>7862</code>

Browser engines

`BROWSER_ENGINE` selects the engine launched by `_get_browser`:

- `chromium` (default) — patchright-patched Chromium. Set `BROWSER_CHANNEL=chrome` to drive a real Google Chrome binary (install it in the image) for a genuine Chrome TLS/version fingerprint; unset uses the bundled Chromium.
- `camoufox` (optional) — a Firefox fork with engine-level fingerprint spoofing. Stronger against fingerprint-based blocks, and unlike a headless-only engine it still renders + screenshots (so `extract`'s Sonnet-vision works). To enable: `pip install '.[camoufox]'` → `python -m camoufox fetch`, add Firefox's system libs (`playwright install-deps firefox`) to the image, then set `BROWSER_ENGINE=camoufox`. On a headless host use `HEADLESS=virtual` (needs `xvfb`) for best stealth, or `HEADLESS=true`.

Neither engine changes the **egress IP**, which is the dominant signal for enterprise CDNs (Akamai et al.): a datacenter IP is denied before the fingerprint is even evaluated. Pair either engine with a non-datacenter IP (residential proxy / self-hosted worker) to actually clear those — on Cloud Run alone they only help fingerprint-gating sites.

Fetch fallback chain

`visit` / `extract` (and a degraded mode of `act`) fetch with a real Chromium first. When a CDN (Akamai, Cloudflare, Imperva) bot-blocks our egress IP and returns a 200-OK "Access Denied" / JS-challenge page, the same URL is re-fetched from different infrastructure, in order:

1. **Tavily Extract** — needs `TAVILY_API_KEY`; different egress IP, fast.
2. **Anthropic** `web_fetch` — server-side fetch via the Messages API; reuses `ANTHROPIC_API_KEY`. Server-rendered HTML + PDFs only (no JS).

Results carry a `source` field (`browser` / `tavily` / `anthropic_web_fetch`).

A page that stays blocked after all fallbacks is returned with a `blocked` flag rather than being mistaken for empty content. `act` cannot replay interaction steps through a static fallback, so when the live page is blocked it returns the static fetch with a `degraded` note.

Playbooks (per-domain recipes)

Hard sites get solved once, then the knowledge is cached as a **playbook** so the agent never re-explores. When `visit` / `act` / `extract` / `download_file` / `inspect_network` / `call_api` hits a URL matching a playbook, the result carries a `playbook` field — `strategy`, what to `avoid` (with the reason), an `open_data` source to use instead, the known-good `act_steps`, an `api` recipe (a discovered endpoint + param template to replay with `call_api`), and/or a `proxy` flag (route this domain through the residential proxy — the seeded PIB

entry sets it because Akamai blocks datacenter egress). The agent is told (in the server instructions) to follow it before exploring. The seeded PPAC entries ship with verified `api` recipes (`getGasConsumption`, `getConsumptionPetroleumProductsData`) so those dashboards are a single `call_api` away; the CGA entry routes you to the Monthly Accounts Dashboard `.xls` (skipping the homepage's ASP.NET month-picker postbacks) to read with `download_file(query=...)`; the PIB entry sets `proxy` because Akamai blocks datacenter egress. Two reactive flags compose with it: `blocked` (CDN bot-wall) and `auth_wall` (login/registration gate) → both mean "stop driving the page, use the playbook's open source."

Source of truth — overlay model. The effective list is the in-repo

`playbooks.py` defaults with the GCS object at `gs://$PLAYBOOKS_GCS_BUCKET/$PLAYBOOKS_GCS_OBJECT` **layered on top, keyed by** `id` (hot-reloaded every `PLAYBOOKS_TTL_SECONDS`): the overlay overrides/extends per-id and may add brand-new ids, while ids it doesn't define come straight from code — so a **new code-default playbook auto-surfaces with no re-seed**. When GCS is unset/unreachable, the defaults stand alone. Editing the overlay takes effect **without a redeploy**; the Cloud Run runtime SA needs `roles/storage.objectAdmin` on the bucket.

Admin API (token-gated by `ADMIN_TOKEN`, fail-closed if unset; needs an HTTP transport + a FastMCP build with `custom_route`):

Method	Path	Purpose
GET	<code>/admin/playbooks</code>	current playbooks + <code>source</code> (<code>gcs</code> / <code>default</code>)
PUT	<code>/admin/playbooks</code>	validate + save (body: <code>{"playbooks": [...]}</code> or <code>[...]</code>)
POST	<code>/admin/playbooks/validate</code>	validate without saving (live UI checks)
GET	<code>/admin/playbooks/match?url=...</code>	preview which playbook a URL hits
POST	<code>/admin/playbooks/reload</code>	force a cache refresh from GCS

All require header `X-Admin-Token: $ADMIN_TOKEN`. **Recommended:** have your platform's admin-settings UI call these via its own backend (server-to-server) so the token never reaches the browser and no CORS is needed. (The left-sidebar admin UI itself lives in the platform repo, not here.)

Stack

Layer	Library
Browser engine	<code>patchright</code> (patched Playwright)
Structured extraction	Anthropic Claude Sonnet 4.6, with vision input
MCP transport	<code>mcp[server]</code> FastMCP — stdio / SSE / streamable-http
Session isolation	One Playwright context per MCP <code>client_id</code>